High-level Digital Design

Lab Assignment 4

Design of a radix-4 Booth multiplier

0. Goals

The goal of this laboratory assignment is the design of a 16-bit x 16-bit multiplier based on the radix-4 Booth algorithm. The multiplier will be developed with an interface that will permit to construct later an AXI4 peripheral that can be managed by the HPS by means of interrupts. The description of the multiplier has to be done using the SystemVerilog hardware description language. First, the principle of the Booth multiplication algorithm will be explained, followed by a detailed specification of the system to be developed.

1. The Booth multiplication algorithm

The multiplication of two numbers M and N represented respectively by bit strings of length m and n is carried out using the classical multiplication algorithm which consists of multiplying the multiplicand by each digit of the multiplier and then adding the obtained partial products shifted appropriately. Figure 1 shows as an example the multiplication of two 4-bit numbers.

```
      B = 1 1 0 1 (13)
    x A = 1 1 1 0 (14)
        -----------
          0 0 0 0
        1 1 0 1
      1 1 0 1
  +   1 1 0 1
  -------------
P =1 0 1 1 0 1 1 0 (182)
```

Figure 1. Example of a 4-bit x 4-bit digital multiplication.

As you can see, when binary digits must be multiplied, the multiplication function is reduced to the logical AND function. Therefore, for the implementation of the multiplication operation, it will be necessary to use AND functions and adders as basic primitives. Note that if the multiplicand and multiplier are represented by bit strings of length m and n, the result of their product will be represented by a string of length n + m.

One way to carry out digital multiplication is to generate in parallel all the AND functions between the bits of the multiplicand and the multiplier, and then to carry out the sum of these functions according to the scheme shown in figure 1. This alternative represents the parallel implementation of multiplication, and is the fastest way to carry out the operation, although it requires a considerable area as the number of bits grows.

If speed is not the most important requirement for implementing the multiplication, a sequential implementation can be carried out. The strategy consists in obtaining in each clock cycle the product of one digit of the multiplier by all the digits of the multiplicand (one of the rows shown in figure 1). This partial product will be shifted and then added to the previous partial product. Therefore, after n clock cycles it will be possible to obtain the result of the product. This implementation of the multiplication is commonly

called the parallel-series strategy, and the algorithm used to implement it is called the Robertson algorithm. The fundamental advantage of this implementation compared to the fully parallel one is given by the fact that it only needs m AND functions (n x m in the parallel version) and an adder of m bits (n in the case of the parallel version) for its implementation. Figure 2 shows a possible realization of this multiplication algorithm in case it is desired to multiply two 8-bit numbers. It is worth noting that sout for multiplicand A is taken from the least significant bit of the shift register, whose value is shifted right each clock cycle. As can be seen in the figure, each clock cycle one bit of the product (p_i) will be obtained and transferred to a shift register, and after 8 clock cycles the shift register located at the bottom of the figure will contain the 8 most significant bits of the result, and together with the 8 bits generated previously (stored in an additional 8-bit shift register) will constitute the 16 bits of the final product.

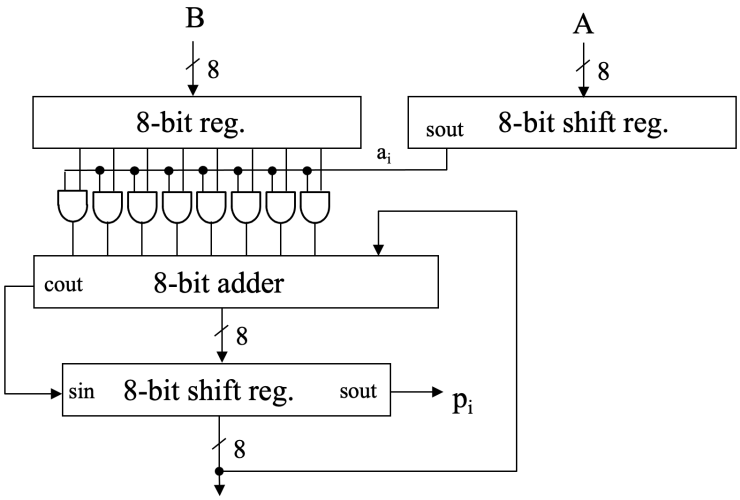

Figure 2. Principle of the parallel-series multiplication.

An improvement over the parallel-serial implementation of multiplication is what is called the Booth's algorithm. This algorithm reduces the number of addition operations to be performed to implement a multiplication operation in the case that the multiplier is made up of strings of bits with a value '1'.

If we assume that the multiplier contains a string of bits with a value '1' starting at position i and ending at position j-1, then the factor by which the multiplicand will be multiplied will be $2^{j-1} + 2^{j-2} + ... + 2^{i-1} + 2^i$. However, by doing simple algebraic manipulations it can be shown that this factor is in fact equal to $2^j – 2^i$.

That is, suppose that the multiplier is represented by 4 digits and that its value is 0111 (7 in decimal, the string starts at position 0 and ends at position 2). It is clear that this value can also be reached as $2^3 – 2^0 = 8 – 1 = 7$. Therefore, the algorithm will perform a sequential inspection of the multiplier bit by bit, and with each clock cycle it will shift the result by one position (equivalent to multiply by 2 the multiplier). In this way, the only actions that will need to be done in order to implement the multiplication will be to subtract the multiplicand when the beginning of a chain of 1s is detected and to add the multiplicand when its end is detected.

Table 1 shows the actions that will need to be taken depending on the values of the current bit (m_i) and the previous bit (m_{i-1}) of the multiplier in a specific step of the algorithm.

m_i	m_{i-1}	Action
0	0	None (out of a string)
0	1	Add multiplicand (end of string)
1	0	Subtract multiplicand (start of string)
1	1	None (inside a string)

Table 1. Actions to be taken by the Booth algorithm.

Figure 3 shows a functional scheme of the implementation of this algorithm. Note that this scheme assumes, as it will be the case for the design to be completed, that the numbers to be multiplied are represented in two's complement, in which case the complement of a number implies to invert all its bits and add a '1'.

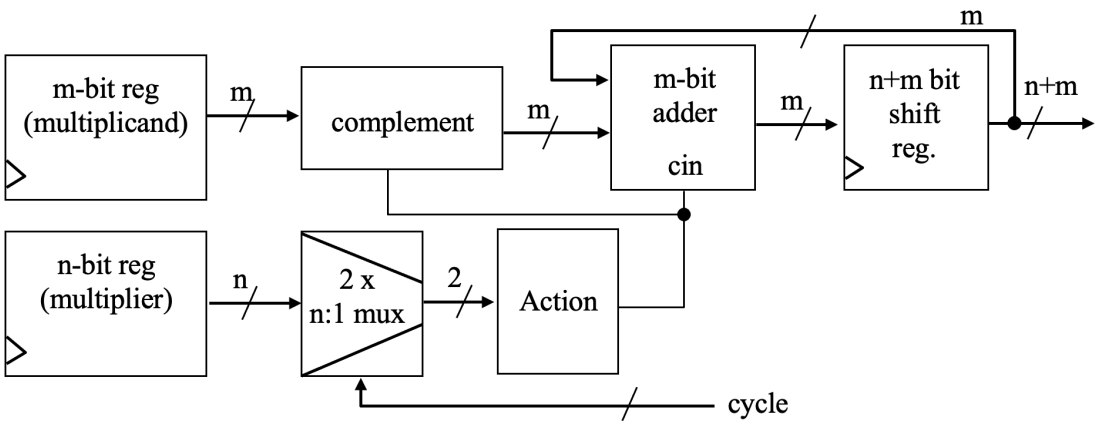

Figure 3. Functional block diagram of a Booth multiplier.

An improvement over the basic Booth algorithm (also called radix-2 Booth algorithm) is what is called the radix-4 Booth algorithm. This algorithm speeds up the multiplication process by simultaneously inspecting 2 bits of the multiplier and shifting (in both the inspection window and the multiplicand) two bits per clock period. In this way, the number of clock cycles required to perform a multiplication is reduced to the number of bits in the multiplier divided by two. Given that two-bit shifts are now made, the actions to be performed with the multiplicand will be expanded, so that it will now also be possible to add or subtract the multiplicand multiplied by 2 (that is, shifted to the left one position). Table 2 shows the set of actions to be performed, taking into account that m_{i+1} and m_i are the two bits of the multiplier being inspected at a particular stage.

m_{i+1}	m_i	m_{i-1}	Action
0	0	0	None
0	0	1	Add multiplicand
0	1	0	Add multiplicand
0	1	1	Add 2 x multiplicand
1	0	0	Subtract 2 x multiplicand
1	0	1	Subtract multiplicand
1	1	0	Subtract multiplicand
1	1	1	None

Table 2. Actions for the radix-4 Booth multiplier.

2. Specification of the design

It is asked to design a 16-bit x 16-bit radix-4 Booth multiplier. The multiplier will have two 16-bit inputs, called *data_a* and *data_b*, and one 32-bit output, called *result*. An additional input, called *start*, will indicate, with a rising edge, that the data inputs are already valid and the multiplication may start. While the multiplication is in progress, the multiplier will activate (at high level) an output called *busy*. When the multiplication has completed the multiplier will activate (at high level) an output called *irq*. This output will be later used as an interrupt request input for the HPS of the Cyclone V device. When an input called *ack* is active (at high level) the outputs *busy* and *irq* will be deactivated. This input will be provided later by the HPS and will serve to clear the interrupt generated by the multiplier. The multiplier will have a clock input, called *clk*. This will be connected to a clock signal with a frequency of 100 MHz. Additionally, it will have a synchronous reset input, active low, called *resetn*. Figure 4 provides an illustrative timing sequence of the functionality of the multiplier.

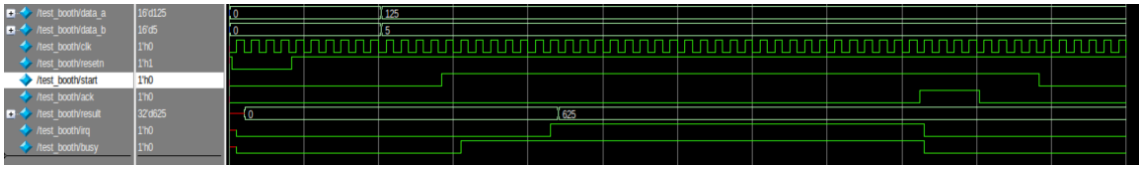

Figure 4. Timing sequence of the functionality of the multiplier.

Therefore, the interface of the SystemVerilog module to be designed should be the following:

```
module booth(clk, resetn, start, busy, irq, ack, data_a, data_b, result);


input clk;                  // global clock signal, 100 MHz frequency
input resetn;               // global reset signal, active low
input start;                // signal that activates the multiplication process
                            // by a rising edge
output busy;                // output that indicates that a multiplication
                            // process is in progress
output irq;                 // IRQ signal activated when the multiplication has
                            //completed
input ack;                  // Input used to deassert the IRQ and busy outputs
input [15:0] data_a;        // First 16-bit operand
input [15:0] data_b;        // Second 16-bit operand
output [31:0] result;       // result of the multiplication
```

In this laboratory assignment it is asked to create the functional description of the multiplier and to carry out its functional simulation using QuestaSim. Therefore, you should:

1. Create a folder called *lab4* and go to it with a terminal.

2. Run QuestaSim by running the command *vsim*.

3. Once the QuestaSim interface opens, it is necessary to create a new project. From its main menu, select the option File -> New -> Project… In the dialog window that appears now, give a name to the project and the click on the *OK* button.

4. In the next dialog window you can either create a new file or import an existing file if you have already created it.

5. For the compilation and simulation process within QuestaSim you should follow the procedure explained in laboratory assignment 3.

Note: Since the multiplier works with numbers provided in 2's complement format, you should carry out a simulation considering the four possible combinations of sign (i.e., ++, +-, -+, --) for multiplicand and multiplier to make sure that it works properly for any combination of signs. Within the *Wave* timing diagram of QuestaSim, you should select Radix -> Decimal for the signals that are to be interpreted in 2's complement.

Opional part: As an optional improvement to the design, you can include an additional input, called *irq_enable*. This input will enable, at high level, the activation of the *irq* output of the multiplier. If this input has a value '0' the multiplier will not activate its *irq* output.